Problem: Assembly ist zu komplex

- schwer zu lesen und nachzuvollziehen
- schwer zu schreiben
  - ohne Fehler zu machen…
- schwer zu warten
- viel Wiederholung von Syntax

```
mult():
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-4], 3
        mov     DWORD PTR [rbp-8], 2
        mov     DWORD PTR [rbp-12], 0
        jmp     .L2
.L4:
        mov     edx, DWORD PTR [rbp-4]
        mov     eax, edx
        sar     eax, 31
        shr     eax, 31
        add     edx, eax
        and     edx, 1
        sub     edx, eax
        mov     eax, edx
        cmp     eax, 1
        jne     .L3
        mov     eax, DWORD PTR [rbp-8]
        add     DWORD PTR [rbp-12], eax
.L3:
        sar     DWORD PTR [rbp-4]
        sal     DWORD PTR [rbp-8]
.L2:
        cmp     DWORD PTR [rbp-4], 0
        jne     .L4
        mov     eax, DWORD PTR [rbp-12]
        pop     rbp
        ret
```

Variablen

Modulo

If statement

Addition

Bitshift

While loop

Programmiersprachen

Erik Tschöpe

Gliederung

- verschiedene Arten von Programmiersprachen
- wie funktioniert eine Programmiersprache?
- JIT-Kompilierung
- Ausblick

Verschiedene Arten von Programmiersprachen

- interpretiert
- kompiliert
- unterschiedlich schwer zu lernen

Programmierparadigmen:

- Objektorientierung -> Alles ist ein Objekt, selbst die Zahl 3
- funktionale Programmierung -> Funktionen sind auch “Objekte”, die man in Variablen speichern kann

Kompilierte Programmiersprachen

Programmcode -> Compiler -> Maschinenanweisungen (Assembly)

Vorteile	Nachteile
- Performance - direkter Zugriff auf Hardware - keine Runtime	- hardwarespezifisch - strengere Syntax - manuelles Memory Management - bei Fehlern im schlimmsten Fall zum Absturz

- immer statische Typisierung
- Anwendung: z.B. Betriebssysteme
- z.B. C, Rust

Einschub: Memory Management

Stack

- wie ein Stapel Teller: oben kommt etwas drauf, oben nimmt man es wieder weg
- wird automatisch vom Programm verwaltet
- für kurze, kleine Daten gedacht
- sehr schnell

Heap

- wie ein Lagerraum mit freien Plätzen
- das Programm muss Platz selbst anfordern und wieder freigeben
- für größere oder länger genutzte Daten
- flexibel, aber etwas langsamer

Interpretierte Programmiersprachen

Programmcode wird Zeile für Zeile analysiert und ausgeführt

Vorteile	Nachteile
- automatisches Memory Management - Kompatibilität - Fehlertoleranz	- langsamere Ausführung - Interpreter muss installiert sein (Runtime)

- Dynamische Typisierung
- Fehlertolerant

z.B. Python, Javascript

Einschub: Java

- teils kompiliert, teils interpretiert
- Java Code wird zu Jvm Bytecode kompiliert
- Jvm Bytecode wird von Jvm interpretiert

Vorteil:

- Java läuft überall wo es eine Jvm gibt
- “Write once, run everywhere”

Syntax

- “Grammatik” der Programmiersprache
- bestimmte Schlüsselwörter
- für jede Programmiersprache individuell

```rust
fn main(){
    let x: u32 = 1;
    let y: u32 = 3;
    let z: u32 = x + y;
    println!("x + y = {}",z)
}
```

```java
public class Main {
    public static void main(String[] args) {
        int x = 1;
        int y = 3;
        int z = x + y;
        System.out.println("x + y = " + z);
    }
}
```

```python
x = 1
y = 3
z = x + y
print("x + y = ", z)
```

Wie funktioniert eine Programmiersprache?

so einfach wie möglich…

Lexer

- Quellcode wird in Tokens unterteilt

Quellcode	Tokens
let x = 42; //variable x auf 42 setzen	KEYWORD(let) IDENTIFIER(x) EQUALS NUMBER(42) SEMICOLON

Parser

- Token werden in Kontext gesetzt
- Baumartige Struktur (Abstract Syntax Tree)

```
x = 9 / (1 + 2)
```

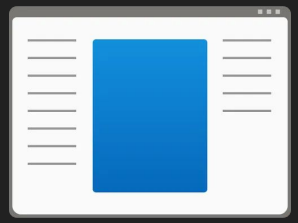
→

```
IDENTIFIER(x)
EQUALS(=)
NUMBER(9)
DIVIDE(/)
LEFT_PAREN(()
NUMBER(1)
PLUS(+)
NUMBER(2)
RIGHT_PAREN())
```

→

```
   Assignment
    /      \
Var(x)     Divide
          /      \
       Num(9)    Add
                /   \
             Num(1)  Num(2)
```

Interpreter/ Codegen



- Wertet AST aus -> von unten nach oben

Kompilierte Programmiersprachen	Interpretierte Programmiersprachen
AST wird in Binärcode übersetzt Ausführbare Datei wird erzeugt (.exe)	AST wird Stück für Stück ausgeführt

Just in time (JIT) Kompilierung

- Programmiersprache wird Ausdruck für Ausdruck übersetzt
- nah an der Performance von kompilierten Programmiersprachen
- flexibilität von interpretierten Programmiersprachen

-> Kombination aus interpretiert und kompiliert

z.B. Lua, JavaScript in Chrome

Warum gibt es **so viele** Programmiersprachen?

- komplexe Probleme die neue Lösungen fordern

Wird es irgendwann nur noch eine Programmiersprache geben?

ChatGPT:

- unwahrscheinlich, da jeder Anwendungsbereich andere Anforderungen hat
- ständige Weiterentwicklung
- persönliche Präferenzen -> Syntax

Fragen?

Ich vergesse immer irgendwas…

Danke fürs Zuhören

… und an Justus für die Vorarbeit

Quellen

https://www.freecodecamp.org/news/compiled-versus-interpreted-languages/
https://www.geeksforgeeks.org/compiler-design/difference-between-compiled-and-interpreted-language/
https://www.ibm.com/docs/en/zos-basic-skills?topic=zos-compiled-versus-interpreted-languages
https://www.geeksforgeeks.org/java/just-in-time-compiler/
https://accu.org/journals/overload/26/145/balaam_2510/
https://blog.gopenai.com/understanding-how-parsers-work-writing-my-own-programming-language-fe1232ee6253
https://www.geeksforgeeks.org/dsa/stack-vs-heap-memory-allocation/

Und eine Menge Wissen aus meinem Schädel, das ich über die letzten Jahre gesammelt habe